qa-mask-bench: KV-Cache Masking Strategies for Batched Multi-Question QA

June 2, 2026

1 Setup

Each **round** has 6 self-contained `(info, question, answer)` items (fictional facts, so an answer lives in exactly one document). An **ask** poses 2 of the round's questions at once; their two documents sit at random positions among the cached six (random KV access). We sample 5 asks per round over 5 rounds = 25 asks = 50 questions per case, on `Qwen2.5-72B-Instruct-AWQ` (KV page size 16, greedy). Each `(round, case)` runs once, so the base cache is built once and reused across the round's asks. We sweep three input sizes — **base**, **5**×, **30**× — with identical facts/asks, inflating the documents and system prompt with neutral filler.

2 Strategies

#	Strategy
1	all-cached: system + all 6 infos cached; attend to all.
2	mask+drop (cascade): infos in page-aligned slots; mask + `drop` unused per ask.
3	mask+drop (exclusive): as #2, each info prefilled attending only to system.
4	inline: only system cached; the 2 used infos prefilled inline each ask.
5	pure-mask (cascade): infos packed contiguously; mask unused, *no drop*.
6	pure-mask (exclusive): as #5, exclusive prefill.

3 Token counts per part (per ask)

Dataset	System	Used info (2)	Query	Output	Cached base
base	93	166	58	∼17	590
5×	467	823	58	∼17	2936
30×	2815	4874	58	∼17	17437

Table 1: *Used info* is the two documents an ask needs. *Cached base* = system + all 6 infos (prefilled once per round in cases 1/2/3/5/6; case 4 caches only the system). Query/output are independent of document size.

4 Results

base

Case	EM	F1	Prefill (ms)	Decode (ms)	Reg (ms)	Attend	Resident
1 all-cached (cascade)	**1.000**	**1.000**	60	493	664	648	648
2 mask+drop (cascade)	0.660	0.660	59	495	877	317	340
3 mask+drop (exclusive)	0.660	0.660	59	495	878	317	340
4 inline	**1.000**	**1.000**	195	493	94	317	317
5 pure-mask (cascade)	**1.000**	**1.000**	60	494	661	317	648
6 pure-mask (exclusive)	0.700	0.716	60	497	660	317	648

5×

Case	EM	F1	Prefill (ms)	Decode (ms)	Reg (ms)	Attend	Resident
1 all-cached (cascade)	**1.000**	**1.000**	65	506	2521	2991	2991
2 mask+drop (cascade)	0.640	0.648	62	500	2764	1348	1373
3 mask+drop (exclusive)	0.640	0.648	62	500	2767	1348	1373
4 inline	**1.000**	**1.000**	714	498	392	1348	1348
5 pure-mask (cascade)	**1.000**	**1.000**	65	508	2523	1348	2991
6 pure-mask (exclusive)	0.680	0.696	66	508	2529	1348	2991

30×

Case	EM	F1	Prefill (ms)	Decode (ms)	Reg (ms)	Attend	Resident
1 all-cached (cascade)	**1.000**	**1.000**	94	562	15942	17491	17491
2 mask+drop (cascade)	0.600	0.616	74	527	16603	7748	7770
3 mask+drop (exclusive)	0.600	0.616	74	525	16601	7748	7770
4 inline	**1.000**	**1.000**	**4233**	527	2252	7748	7748
5 pure-mask (cascade)	**1.000**	**1.000**	94	563	15968	7748	17491
6 pure-mask (exclusive)	0.700	0.708	94	566	16172	7748	17491

Table 2: 50 questions per case; latency averaged per ask. Prefill = TTFT; Reg = once-per-round base build; Attend / Resident = tokens attended / physically in the KV at decode.

5 How the picture changes with input size

6 Findings

- **The masking decode saving appears only at scale.** Decode is weight-bound for small caches (base/5×: all cases ≈ 493–508 ms), so masking buys nothing. By 30× (∼17.5k cached tokens) attention starts to matter: full-KV cases (#1/#5, ∼17.5k resident) decode in 562 ms vs 527 ms for the masked-and-dropped cases (∼7.7k resident) — a ∼6% saving that grows with context length.
- **Inline prefill becomes prohibitive.** It reprocesses the two used documents every ask, so

Metric	base	5×	30×
Cached base (tokens)	590	2936	17437
Decode, full KV (#1, ms)	493	506	562
Decode, masked KV (#2, ms)	495	500	527
Masking decode saving	∼0%	∼1%	**∼6%**
Inline prefill (#4, ms)	195	714	**4233**
Cached prefill (#1/5, ms)	60	65	94
mask+drop accuracy (EM)	0.66	0.64	0.60
pure-mask accuracy (EM)	1.00	1.00	1.00

Table 3: Trend across input sizes. The masking decode saving grows from nil to ∼6%; inline prefill explodes (22×); mask+drop accuracy erodes while pure-mask stays perfect.

its TTFT scales with document size: 195 ms → 714 ms → 4233 ms (22×), versus a flat ∼60–94 ms for the cached strategies (which prefill only the question).

- **"mask+drop" corrupts multi-token answers, worse at scale** (EM 0.66 → 0.64 → 0.60). RoPE is baked into the keys before they enter the cache, so the page-aligned padding required for dropping inserts positional gaps that scramble word boundaries (names break, numbers survive); larger documents mean larger gaps:

```
gold: 'Doran Drummel' 'Karsil Hallow'
case 2: 'Dorandrummel' 'Kars Silhallow' (mask+drop) BAD
case 5: 'Doran Drummel' 'Karsil Hallow' (pure-mask) OK
```

- **A real accuracy/decode tension emerges at scale.** Pure masking (#5) keeps positions contiguous, so accuracy stays 1.000 — but it does *not* drop the masked pages, so its decode (563 ms) matches the full-KV case. The ∼6% decode saving requires dropping, which is exactly what corrupts the answers. With keys pre-rotated at prefill, you cannot have both cheaply.

- Cascade prefill beats exclusive (1.00 vs 0.70 under pure masking) at all sizes. Base-cache build (Reg) scales with total cached tokens but is amortized over the round's asks.

7 Analytical estimate: how long must the prompt be to save 50% decode?

The three input sizes let us fit a simple model for per-token decode latency as a function of the resident KV length N:

$$d(N) = W + \alpha N, \tag{1}$$

where W is the fixed, KV-independent (weight-bound) cost and α is the per-token attention cost added by each resident KV token. Fitting the three full-KV (#1) measurements (N, d) in μs — (648, 31659), (2991, 32433), (17491, 36054) — gives

$$W \approx 31\,572\,\mu\text{s/token}, \qquad \alpha \approx 0.257\,\mu\text{s/token per KV token}, \tag{2}$$

which reproduces all three points to within 0.3% (and the masked-case decode to within ∼1%). The KV-attention share of a decode step is $\alpha N/(W + \alpha N)$: just ∼0.5% at N=648 (base), but ∼13% at N=17.5k (30×).

Saving from masking. An ask caches n=6 documents of I tokens plus an s-token system prompt and attends to only k=2, so masking drops $\Delta N = (n-k)\,I$ tokens. The fractional decode saving is

$$\sigma(N) \;=\; \frac{d(N_{\text{full}}) - d(N_{\text{keep}})}{d(N_{\text{full}})} \;=\; \frac{\alpha\,\Delta N}{W + \alpha N_{\text{full}}}, \qquad N_{\text{full}} = s + nI. \tag{3}$$

This formula predicts 0.3% / 1.3% / 6.9% at base / 5× / 30×, matching the measured ∼0% / ∼1% / ∼6%. As I grows the system prompt becomes negligible and $\Delta N \to \frac{n-k}{n} N_{\text{full}} = \frac{2}{3} N_{\text{full}}$, so the *maximum* achievable saving is capped at $\frac{2}{3} \approx 67\%$ (you can only drop the 4 of 6 unused documents).

Solving $\sigma = 0.5$. In the large-I limit $\sigma = \frac{2}{3}\alpha N/(W + \alpha N)$. Setting this to 0.5 gives

$$N^{\star} \;=\; \frac{3W}{\alpha} \;\approx\; \frac{3 \times 31572}{0.257} \;\approx\; 3.7 \times 10^{5} \text{ tokens}. \tag{4}$$

So the cached prompt (system + all documents) would need to be on the order of **∼370k tokens** — about 21× the 30× cache here, or ∼150 documents of the current ∼2.4k-token size — for masking to halve decode latency. At that point each decode step would cost ≈ 126 ms (∼4× today's), with attention making up ∼75% of it.

Caveats. (i) This is a ∼20× extrapolation beyond the measured range and well past the model's native context window. (ii) It assumes α is constant; if attention becomes memory-bandwidth-bound at very long context, α rises and the 50% crossover arrives *sooner* (smaller $N^{\star}$). (iii) Realising the saving requires `drop`, which — as shown above — corrupts answers via baked-RoPE position gaps; a gap-free drop would be needed to reach 50% savings without the accuracy hit.

Conclusion

Input size does change the trade-off. For small caches, pure masking (#5) strictly dominates — full accuracy at the same latency as caching everything. As inputs grow to ∼30×, two things sharpen: inline prefill becomes prohibitively expensive (22×), and a genuine decode-vs-accuracy tension appears — mask+drop saves ∼6% decode but its baked-RoPE position gaps drag accuracy to EM 0.60, while pure masking holds EM 1.000 at the cost of keeping the full KV resident. The practical recommendation: cache everything or pure-mask (#1/#5) when accuracy matters; only reach for mask+drop in very long-context, latency-critical settings that can tolerate the corruption — ideally after the runtime gains gap-free dropping (e.g. position renumbering).

Reproduce.

```
cd sdk/examples/qa-mask-bench
make prepare # base (scale 1)
python prepare.py --scale 11 --out rounds_large.json # ~5x tokens
python prepare.py --scale 74 --out rounds_30x.json # ~30x tokens
make run CASES=1,2,3,4,5,6 # base
```

```
python run.py --model <M> --cases 1,2,3,4,5,6 --rounds-json rounds_large.json \
    --output results_large.jsonl
python run.py --model <M> --cases 1,2,3,4,5,6 --rounds-json rounds_30x.json \
    --output results_30x.jsonl
python eval.py --results results.jsonl # base
python eval.py --results results_large.jsonl # 5x
python eval.py --results results_30x.jsonl # 30x
```